2018 IEEE International Conference on Software Testing, Verification and Validation Workshops

Risk-Driven Model-Based Testing of Washing Machine Software: An Industrial Case Study

Abdulhadi Kirkici
Vestel Electronics,
Manisa, Turkey
abdulhadi.kirkici@vestel.com.tr

Ceren Şahin Gebizli
Vestel Electronics,
Manisa, Turkey
ceren.sahin@vestel.com.tr

Hasan Sözer
Ozyegin University,
Istanbul, Turkey
hasan.sozer@ozyegin.edu.tr

***Abstract*—We previously introduced an approach for risk-driven model-based testing. In that approach, test models are represented in the form of Markov chains and test case generation is steered by state transition probabilities. These probabilities are iteratively updated based on estimations of risk of failure. Test cases are generated and executed after each model refinement step to detect new faults in less time. The approach was evaluated in the context of two industrial case studies conducted for testing smart TV and smart phone systems. In this paper, we present results of a third case study, where we apply that approach for model-based testing of washing machine software. We define new types of risks for this particular application, which are estimated based on the number of configuration options and power consumption. As a result, the overall test duration was decreased with risk-driven model-based testing although the number of detected faults did not increase.**

***Keywords*-model-based testing, model refinement, statistical usage testing, risk-based testing, industrial case study, software test automation**

I. Introduction

In consumer electronics domain, systems have been transformed from electromechanical systems to software-intensive systems. For instance, smart TV systems comprise tens of millions of lines of code [1]. The size and complexity of this code base are continuously increasing. One the one hand, the reliability of software has become the main designator of overall system reliability. Therefore, software reliability should be high for targeting at a market with high competition. On the other hand, resources are extremely limited in consumer electronics domain due to short time-to-market. It is not feasible anymore to test systems exhaustively. Hence, software testing techniques must be efficient to be able to reveal and fix the most critical faults with limited resources.

Automating test activities [2] is a typical approach preferred for increasing test efficiency. These activities can be related to test case generation, test execution or test oracle [3] implementation. Various techniques can be employed for automating one or more of these activities. One of these techniques is model-based testing (MBT) [4], [5], [6], which is now a state-of-the-practice technique for automating test case generation. Hereby, test cases are generated by systematically exploring various executions paths on a test model, which defines the usage behavior of the system. This reduces the test case creation time. The input test model has to be created manually; however, there have been various techniques proposed to generate [7], extend [8] or refine [9] these models automatically. MBT also facilitates an increased and measured coverage defined based on the test model; however, it turns out to be infeasible to achieve an extensive coverage of the model in practice. For example, a typical test model regarding smart TV systems can comprise thousands of states and transitions [10]. Therefore, the test case generation process must be efficient to explore only those execution paths in the test model that are associated with high risk of failure.

We previously proposed a risk-driven approach [10] to improve MBT efficiency. In that approach, test models are represented in the form of Markov chains and test case generation is steered by state transition probabilities. These probabilities are iteratively updated based on estimations of risk of failure. Test cases are generated and executed after each model refinement step to detect new faults in less time. There are 3 model refinement steps. First, transition probabilities are updated according to frequency of visits at each state, which is estimated based on a collected usage profile. Second, they are updated according to fault likelihood at each state, which is estimated based on static code analysis alerts. Third, they are updated according to error likelihood at each state, which is estimated based on memory leaks observed during previous test executions. The approach was applied in the context of an industrial case study for MBT of a smart TV system [10]. Results were promising, in which new faults were revealed after model refinements. We obtained similar results with a second case study on smart phone systems [11].

In this work, we conducted the third case study, where we apply the approach for risk-driven MBT of washing machine software. We observed that the approach can not be directly applied for these systems since the risk of failure must be defined in different terms. Hence, we adapted two of the refinement steps based on our observations. First, the number of configuration options was more relevant than static code analysis alerts as a risk of fault indicator. Second, fluctuations in power consumption were more relevant than memory profile to estimate the risk of error. We performed risk estimations accordingly and obtained promising results, in which new faults were revealed after model refinements. We also compared these results with those obtained after random-stochastic test case generation without any risk-based model updates.

0-7695-6432-1/18/$31.00 ©2018 IEEE
DOI 10.1109/ICSTW.2018.00080

We observed that the test duration was increased in this case although the number of detected faults did not increase.

The remainder of this paper is organized as follows. In the following section, we summarize related work. In Section III, we introduce the existing approach and its instantiation for testing washing machine software. In Section IV, we explain the case study and present the results. Finally, in Section V, we conclude the paper with a discussion of future work.

II. Related Work

There have been various MBT techniques proposed [5]. These techniques employ a varity of formalisms to document a test model like finite state machines [12], [13]. In our approach, test models are specified in the form of Markov chains [14].

MBT is now a state-of-the-practice technique actively being used in industry. There have been several case studies [15], [16] reported for evaluating MBT in an industrial setting. In our work, we evaluate the effectiveness of our approach in an industrial setting as well. In particular, we focus on the consumer electronic domain.

Our approach basically involves iterative refinements of a test model used for MBT. There have been many other approaches [17], [8], [18], [10] that are also based on iterative model refinements. These studies differentiate from our approach by their goal of extending test models. They augment existing models with additional states and transitions to incorporate new system behavior. This behavior is mainly discovered by monitoring the system execution at runtime. In our approach, we do not aim at extending existing test models. We assume that the model is complete enough and it is so large that it is not possible to achieve an extensive coverage of the model during test case generation. We only add fault/error states and update transition probabilities to steer the test case generation process on various execution paths on the model. We employ multiple types of analysis for this purpose based on usage profile, static analysis as well as dynamic analysis.

We have previously proposed a risk-based MBT approach [19] inspired from the principles of risk-based testing [20]. The risk estimation in that approach was only based on dynamic analysis focusing on memory leaks. We extended that approach later with the use of usage profile and static analysis for risk estimation [10]. Hereby, dynamic analysis was again focusing on memory leaks. Usage profile was used for calculating the probability that a particular feature of the system is used and associated faults gets triggered. Static code analysis alerts were used to estimate the likelihood of faults regarding each feature. In this paper, we present results of a third case study, where the subject system is washing machine software. We define new types of risks for this particular type of system, which are estimated based on the number of configuration options and power consumption.

III. Risk-Driven MBT Approach

The generic risk-driven MBT approach is depicted in Figure 1. Initially, we use an existing test model for generating

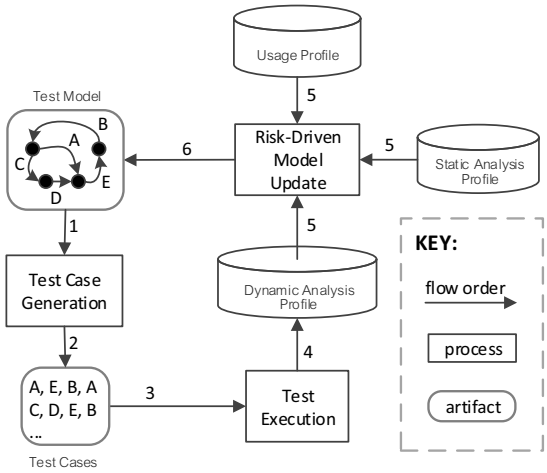

Fig. 1: The generic risk-driven MBT approach.

test cases with a MBT tool. These test cases are executed on the system. In the initial test model, all the probability values for outgoing transitions of a state are equal to each other, adding up to 1. Then, these values are updated based on risk estimations. The updated test model is used for test case generation and execution once again. Hereby, the test case generation algorithm should follow the principles of statistical usage (or risk-based) testing, in which the model is explored randomly based on the probability values associated with transitions. The cycle of risk-driven model update, test case generation and test execution is repeated multiple times. The update in each iteration is performed based on either usage profile, static analysis or dynamic analysis. Hereby, the types of analysis and the way that risk estimation is performed can differ among systems. For instance, dynamic analysis can be performed to obtain memory profile, energy consumption, frequency of interaction or possibly any other attribute regarding software modules to estimate risk. In the following, we introduce an instance of this approach for MBT applied on washing machine software (See Figure 2) and we explain each model refinement step in detail.

The first two iterations (*Iteration 0* and *Iteration I*) are in fact the same as the previous implementations of our approach [10], [11]. We can see in Figure 2 that *Iteration 0* just involves the use of the existing test model for MBT as is. *Iteration I* is performed based on a previously collected usage profile. Hereby, probability values associated with each outgoing transition of a state is updated to be proportional to the ratio of visits made to its target state [10]. Let's say, there are n outgoing transitions from a state s to other states enumerated from 0 to $n-1$. If the number of visits to each of these states is calculated as v_i according to the usage profile, then the probability value assigned for transition reaching to state i is updated as $v_i / \sum_{i=0}^{n-1} v_i$.

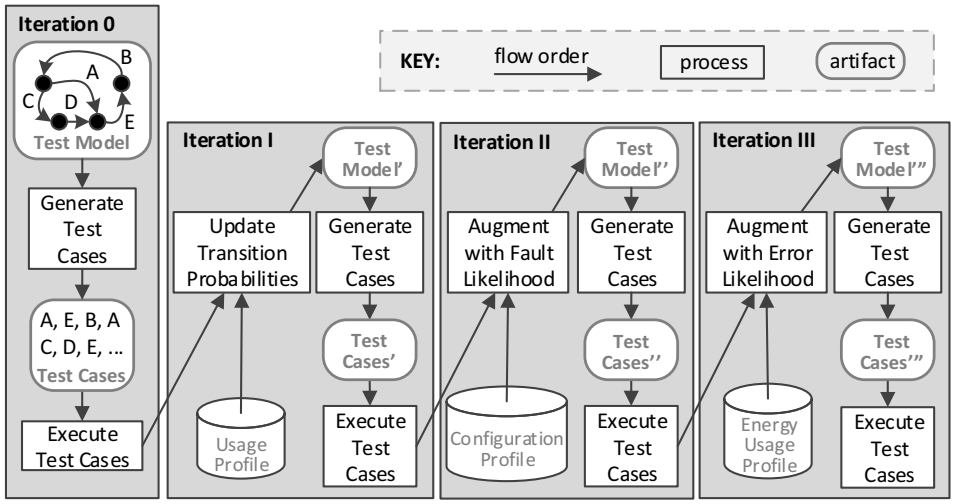

Fig. 2: The instance of the approach applied for MBT of washing machines.

In *Iteration II*, we perform an update based on static analysis. In the previous two implementations of our approach [10], [11], we utilized static code analysis alerts to estimate the risk of fault. However, for this implementation, our discussion with the domain experts revealed that the code base used for washing machine systems is stable for the most part and it is not subject to many alerts. On the other hand, the configuration space of software is very large and the most of the faults are introduced due to wrong/conflicting settings of configuration options. Hence, we defined and calculated the risk of fault at a state as the ratio of configuration options associated with that state. Then, the model is updated according to this risk [10]. First, a new state, E is added to the model. Second, new transitions are added from each state to E. Third, the probability value for transition from a state s to E is set as the ratio of configuration options associated with s to the total number of configuration options. Finally, the probability value for each of the other outgoing transitions from s is multiplied by $(1 - p)$. This final update makes sure that the probability values for all the outgoing transitions of s add up to 1.

In *Iteration III*, we perform an update based on dynamic analysis. In the previous two implementations of our approach [10], [11], we utilized memory profile to estimate the risk of error. However, for this implementation, our discussion with the domain experts revealed that memory usage for washing machine systems is usually stable. On the other hand, fluctuations in power consumption are more relevant indicators of an error. Hence, we defined and calculated the risk of error at a state to be proportional to the amount of power consumption associated with that state. Then, the model is updated just like in *Iteration II* by adding a new state and updating probability values on transitions.

In the following, we introduce a case study and illustrate each step of the approach. We also share and discuss the results obtained.

IV. Industrial Case Study

In this section, we introduce a case study from the consumer electronics domain. In particular, we will illustrate the application of our approach for MBT of washing machine products developed by Vestel[1], which is one of the largest home appliances manufacturers in Europe. Washing machine failures can create serious damages due to high volume of water, which can lead to a flooded laundry room.

We used a test model that was previously developed by the software test group in the company. The model is specified in the form of a Markov chain with MaTeLo[2] that is used as the MBT tool. Figure 3 depicts the model. Hereby, states mainly represent different programs of the washing machnine, including *Cotton*, *Easycare*, *Wool-30*, *Rinse*, *Spin*, *Handwash*, *Sportswear*, *Mix-30*, *Duvet*, *Blouses*, *Daily-60*, *Rapid-15*, *Babycare*.

We used two of the test case generation algorithms provided by MaTeLo. The first one is called *Minimum Arc Coverage* algorithm [21], which terminates after all the transitions in the model are visited at least once. This algorithm is used only for *Iteration 0*, where all the probability values for outgoing transitions of a state are equal to each other. The second algorithm is called *User Oriented* algorithm [21], which is used after each model refinement step. This algorithm traverses the model by choosing a transition at each state non-deterministically based on transition probabilities. This

[1]http://www.vestel.com.tr
[2]http://www.all4tec.net

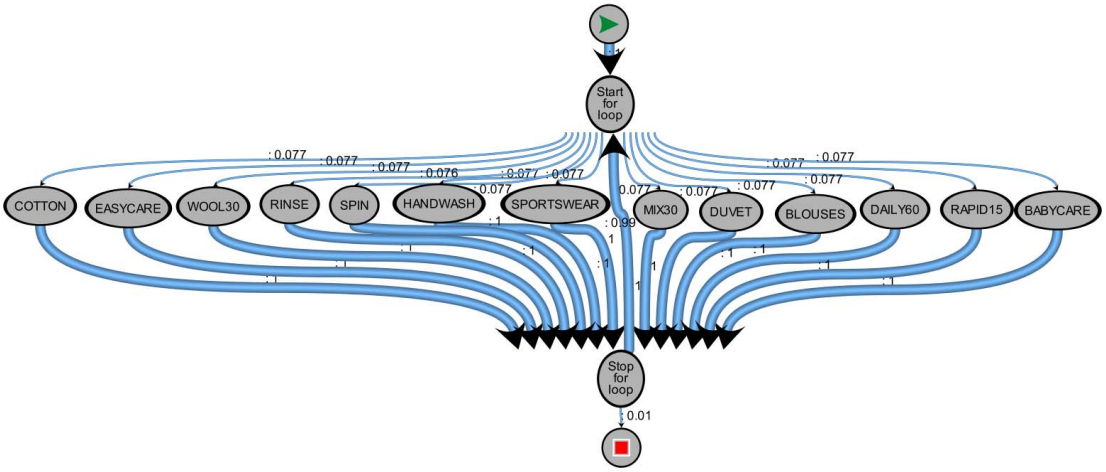

Fig. 3: An initial test model used for MBT with default probability values.

algorithm has two parameters; the maximum number of test steps for a test case, and the maximum number of test cases, which are set as 5000 and 1, respectively.

In total 248 test steps were generated in *Iteration 0*. Execution of these test steps took 84 hours. 8 faults were detected during test execution.

The test model is updated in 3 successive iterations as described in the previous section. We used the toolset[3] that we previously developed to perform model updates automaticaly [11]. In the following subsections, we describe these iterations and updates.

A. Model Refinement based on Usage Profile

In *Iteration I*, the test model is updated based on usage profile. To collect this profile, 8 products were sent to 8 field testers. These products are connected to the Internet through which log files are shared to record programs being used. These files are collected for 100 days to analyze the frequency of usage for each program. Then probability values on transitions are updated as listed in the first part of Table I. 244 test steps were generated after the update. Execution of these test steps took 82 hours. 5 faults were detected during test execution. 4 of them were previously found. Therefore, 1 new fault was identified after the first iteration.

B. Model Refinement based on Static Analysis

In *Iteration II*, we utilized number of configurations in the programs of the washing machine. The second part of Table I lists the number of configurations for each program and the calculated probability values accordingly. 115 test steps were generated after the model is updated based on these values. Execution of these test steps took 39 hours. 6 faults were detected during test execution. 4 of them were previously found. Therefore, 2 new faults identified after the second iteration.

[3]The toolset is available online at the project Web page: http://srl.ozyegin.edu.tr/projects/armor/

C. Model Refinement based on Dynamic Analysis

In *Iteration III*, we utilized power consumption measured during the execution of each program. The third part of Table I lists the amount of power consumption associated with each program and the calculated probability values accordingly. 67 test steps were generated after the model is updated based on these values. Execution of these test steps took 22 hours. 3 faults were detected during test execution. 2 of them were previously found. Therefore, 1 new fault was identified after the third iteration.

D. Results and Discussion

The overall results are summarized in Table II. We were able to detect 8 faults with the original test model. This model is transformed in successive iterations as can be seen in Figure 4. We were able to detect 4 more new faults during these iterations. These faults caused failures related to stability issues in the platform such as door lock, spin, drain and shaking problems. It took 227 hours (84+82+39+22) to execute all the test cases and 12 faults were found (8+1+2+1) in total.

We compared the obtained results with those obtained after random-stochastic test case generation without any risk-based model updates. Hereby, we just generated and executed test cases by using the same test model. We repeated this process 4 times to match the total number of iterations with our approach. The overall results are listed in Table III. We can see that the results regarding *Iteration 0* are the same as those listed in Table II. This is expected because the same test model is used in both cases for this iteration. We can also see that 12 distinct faults were found (8+3+1) in total.

Although the number of distinct faults detected is the same for both approaches, there is a significant difference in terms of test execution time. It took 227 hours to execute all the test cases when the model is updated based on risks. On the other hand, overall test duration was 301 hours when the same model is used for test case generation in successive iterations.

Software Module	Iteration 1		Iteration 2		Iteration 3	
	# of Visits	Calculated Prob.	# of Configurations	Calculated Prob.	Power Consumption (kWh)	Calculated Prob.
COTTON	187	0.142	74	0.138	2.035	0.182
EASYCARE	98	0.075	43	0.08	2.504	0.225
WOOL30	32	0.024	35	0.065	0.81	0.072
RINSE	15	0.011	23	0.043	0.079	0.007
SPIN	17	0.013	3	0.006	0.079	0.007
HANDWASH	67	0.051	36	0.067	0.14	0.013
SPORTWEAR	56	0.043	40	0.075	0.372	0.034
MIX30	230	0.175	59	0.11	0.325	0.03
DUVET	12	0.01	59	0.11	0.66	0.06
BLOUSES	60	0.046	54	0.101	1.08	0.0096
DAILY60	243	0.185	30	0.056	0.772	0.07
RAPID15	224	0.17	15	0.028	0.074	0.007
BABYCARE	72	0.055	65	0.121	2.202	0.197

TABLE I: Collected data and the corresponding probability values used for updating the test model in successive iterations.

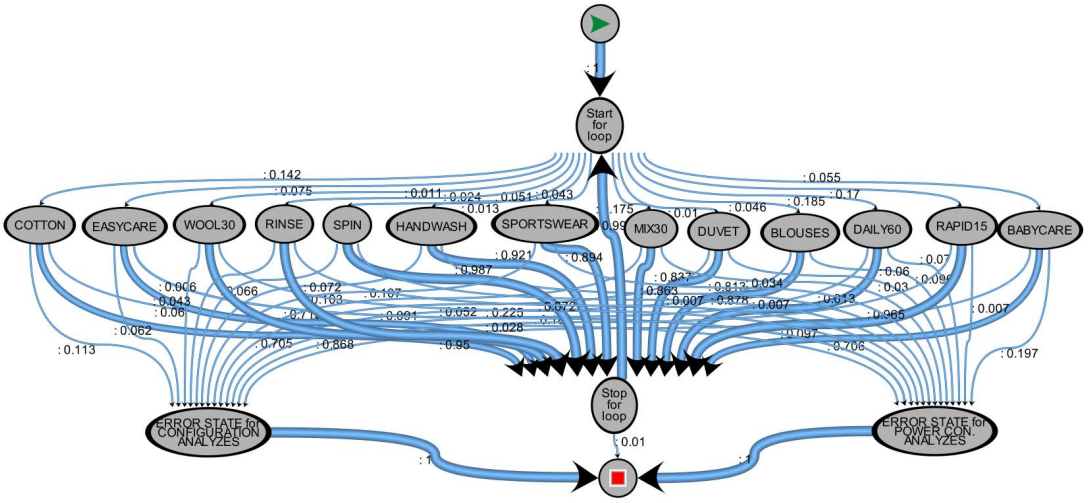

Fig. 4: The updated test model after several iterations.

Hence, test efficiency was decreased when our approach is not applied. Even if we ignore the last iteration during which no new faults were discovered, the total test duration turns out to be 232 hours. This is still 5 hours longer than the total test case execution time in Table II. The adoption of the approach itself is subject to a setup cost. However, this requires only a one-time investment that can be utilized accross projects.

We can observe that the model update based on usage profile did not bring much benefit for this case. However, the first iteration might still be relevant to incorporate the risk of user-exposure of faults. On the other hand, test execution times for the second and third iterations are reduced dramatically (more than 50%). Yet, 3 new faults were detected during these iterations (See Table II). We can also see that the number of test steps were decreased; however, we should note that the number of test steps is not directly proportional to test duration. The execution of some of the test steps (e.g., to complete a particular washing program) can take much more time relative to other test steps.

In general, static analysis can be performed prior to test executions. In this particular case study, for instance, the number of configuration options associated with each state of the model is known. So, in principle, the first two iterations can be merged, where the corresponding risk estimations are reflected to the model directly.

V. Conclusions and Future Work

In this paper, we presented an industrial case study for risk-driven model-based testing of washing machine software. We defined new types of risks for this particular subject system, which are estimated based on the number of configuration options and power consumption. The test model is refined

Iteration #	# of Test Steps	Test Execution Time (hr)	# of Faults Detected	# of New Faults Detected
0	248	84	8	-
1	244	82	5	1
2	115	39	6	2
3	67	22	3	1

TABLE II: Test results after each iteration of risk-driven model-based testing.

Iteration #	# of Test Steps	Test Execution Time (hr)	# of Faults Detected	# of New Faults Detected
0	248	84	8	-
1	250	70	5	3
2	240	78	9	1
3	236	69	4	0

TABLE III: Test results obtained when the same test model is used in successive iterations without any updates.

in successive iterations based on these risk estimations. Test cases are generated and executed after each model refinement step. The goal of executing these test cases is to cover diverse parts of the model and detect faults in less time; especially those faults that are more likely to be exposed to the users as failures. We obtained promising results, in which new faults were revealed after model refinements. We observed that the same set of faults can be discovered when the approach is not applied; however, the overall test duration becomes much longer. As future work, we plan to conduct more case studies focusing on various systems for evaluating the general applicability of the approach.

ACKNOWLEDGMENT

This work is supported by the joint grant of Vestel Electronics and the Turkish Ministry of Science, Industry and Technology (909.STZ.2015). The contents of this article reflect the ideas and positions of the authors and do not necessarily reflect the ideas or positions of the sponsors of the grant. We would like to thank software developers and software test engineers at Vestel Electronics for sharing their code base with us and supporting our case studies.

REFERENCES

[1] U. Yuksel and H. Sözer, "Automated classification of static code analysis alerts: A case study," in *Proceedings of the 2013 IEEE International Conference on Software Maintenance*, 2013, pp. 532–535.

[2] D. Rafi, K. Moses, K. Petersen, and M. Mäntylä, "Benefits and limitations of automated software testing: Systematic literature review and practitioner survey," in *Proceedings of the 7th International Workshop on Automation of Software Test*, 2012, pp. 36–42.

[3] E. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 507 – 525, 2015.

[4] L. Apfelbaum and J. Doyle, "Model-based testing," in *Software Quality Week Conference*, 1997, pp. 296–300.

[5] A. C. D. Neto, R.Subramanyan, M.Vieira, and G. H. Travassos, "A survey on model-based testing approaches: A systematic review," in *Proceedings of the 1st ACM international workshop on Empirical assessment of software engineering languages and technologies*, 2007, pp. 31–36.

[6] J. Boberg, "Early fault detection with model-based testing," in *Proceedings of the 7th ACM SIGPLAN workshop on ERLANG*, 2008, pp. 9–20.

[7] A. Mesbah, A. van Deursen, and D. Roest, "Invariant-based automatic testing of modern web applications," *IEEE Transactions on Software Engineering*, vol. 38, no. 1, pp. 35–53, 2012.

[8] B. Nguyen and A. Memon, "An observe-model-exercise* paradigm to test event-driven systems with undetermined input spaces," *IEEE Transactions on Software Engineering*, vol. 40, no. 3, pp. 216–234, 2014.

[9] C. Gebizli and H. Sozer, "Automated refinement of models for model-based testing using exploratory testing," *Software Quality Journal*, vol. 25, no. 3, pp. 979–1005, 2017.

[10] C. Gebizli, H. Sözer, and A. Ercan, "Successive refinement of models for model-based testing to increase system test effectiveness," in *Proceedings of the 11th Workshop on Testing: Academia-Industry Collaboration, Practice and Research Techniques*, 2016, pp. 263–268.

[11] C. Gebizli, "Automated refinement of models for model-based testing," Ph.D. dissertation, Ozyegin University, Turkey, 2016.

[12] H. Robinson, "Finite state model-based testing on a shoestring," in *Proceedings of the Software Testing and Analysis and Review West Conference*, 1999.

[13] A. Chander, D. Dhurjati, S. Koushik, and Y. Dachuan, "Optimal test input sequence generation for finite state models and pushdown systems," in *Proceedings of the 4th IEEE International Conference on Software Testing, Verification and Validation*, 2011, pp. 140–149.

[14] J. Whittaker and M. Thomason, "A markov chain model for statistical software testing," *IEEE Transactions on Software Engineering*, vol. 20, no. 10, pp. 812–824, 1994.

[15] S. R. Dalal, A. Jain, N. Karunanithi, J. M. Leaton, C. M. Lott, G. C. Patton, and B. M. Horowitz, "Model-based testing in practice," in *Proceedings of the International Conference on Software Engineering*, 1999, pp. 285–294.

[16] J. Keranen and T. Raty, "Model-based testing of embedded systems in hardware in the loop environment," *IET Software*, vol. 6, no. 4, pp. 364–376, 2011.

[17] X. Yuan and A. Memon, "Generating event sequence-based test cases using GUI runtime state feedback," *IEEE Transactions on Software Engineering*, vol. 36, no. 1, pp. 81–95, 2010.

[18] C. Gebizli and H. Sozer, "Improving models for model-based testing based on exploratory testing," in *Proceedings of the 8th IEEE International Computer Software and Applications Conference Workshops*, 2014, pp. 656–661.

[19] C. S. Gebizli, D. Metin, and H. Sözer, "Combining model-based and risk-based testing for effective test case generation," in *Proceedings of the 8th IEEE International Conference on Software Testing, Verification and Validation*, 2015, pp. 1–4.

[20] M. Felderer and I. Schieferdecker, "A taxonomy of risk-based testing," *International Journal of Software Tools and Technology Transfer*, vol. 16, no. 5, pp. 559–568, 2014.

[21] C. Joye, "Matelo test case generation algorithms: Explanation on available algorithms for test case generation," 2018, http://www.all4tec.net/en/automatic-test-generation-strategies.